- More on MVC: https://developer.mozilla.org/en-US/docs/Glossary/MVC

الـ **MVC (Model-View-Controller)** مش مجرد تنظيم ملفات، ده "دستور" تقسيم المهام. تعالى نغطس (Deep Dive) في الفلسفة دي ونعرف كل حتة بتفكر إزاي.

🏛️ إيه هو الـ MVC؟

تخيل إنك فاتح مطعم كبير:

1. **الـ Model (المخزن والمطبخ):** ده اللي فيه الداتا (الخضار واللحمة) وفيه الـ Logic بتاع الطبخ (إزاي بنعمل الأكلة). ملوش دعوة اليوزر شايف إيه.
2. **الـ View (صالة الأكل والمنيو):** ده اللي اليوزر بيشوفه (الـ HTML/EJS). وظيفته يعرض الأكل بشكل حلو، ملوش دعوة الأكل اطبخ إزاي.
3. **الـ Controller (الويتر أو الجرسون):** هو حلقة الوصل. ياخد الطلب من اليوزر، يروح للمخزن (Model) يجيب الداتا، وبعدين يوديها للصالة (View) عشان تتقدم لليوزر.

1️⃣ الـ Model: قلب البيانات والمنطق

الـ Model هو المسؤول الوحيد عن **الداتا**. سواء كانت الداتا دي في Array في الميموري، أو ملف JSON، أو قاعدة بيانات (Database).

- **وظيفته:** بيعرف "شكل" البيانات (Schema) وإزاي بنحفظها (Save) وإزاي بنجيبها (Fetch).
- **القاعدة الذهبية:** الـ Model ميعرفش أي حاجة عن الـ Express أو الـ Request أو الـ Response. هو بيتعامل مع "بيانات" فقط.

2️⃣ الـ View: الواجهة والجمال

الـ View هو اللي بيطلع لليوزر في المتصفح. في حالتنا هو ملفات الـ **EJS** أو الـ **Pug** أو الـ **Handlebars**.

- **وظيفته:** ياخد الداتا "الناشفة" من الـ Controller ويحطها جوه قالب HTML شيك.
- **القاعدة الذهبية:** الـ View مفيهوش Logic معقد. آخره `if` بسيطة أو `loop` لعرض المنتجات. مبيعرفش الداتا جات منين، هو بيستلمها "على الجاهز".

3️⃣ الـ Controller: العقل المدبر

ده اللي إنت لسه باينه دلوقتي. هو اللي بيربط "الراوتر" بالـ "موديل".

- **وظيفته:**
  1. يستلم الـ `req` والـ `res`.
  2. يكلم الـ Model عشان يطلب بيانات (مثلاً `Product.fetchAll()`).
  3. ياخد البيانات دي ويبعتها للـ View عن طريق `res.render()`.
- **القاعدة الذهبية:** هو "المنسق". لو عندك كود "طبخ داتا" كتير، انقله للموديل. لو عندك كود HTML كتير، انقله للـ View. خلي الـ Controller دايمًا "نحيف" (Skinny Controller).

🔄 دورة حياة الطلب (The Request Lifecycle)

عشان تربط الخيوط ببعضها، بص الطلب بيمشي إزاي لما اليوزر يفتح صفحة `/shop`:

1. **Routes:** الـ Router يسمع الطلب ويروح ينادي الفانكشن المعينة في الـ **Controller**.
2. **Controller:** يكلم الـ **Model** ويقوله: "يا موديل، هات لي كل المنتجات اللي عندك".
3. **Model:** يروح يقرأ ملف الـ JSON أو الـ DB، ويرجع الـ Array للـ **Controller**.
4. **Controller:** ياخد الـ Array ده، وينادي الـ **View** ويقوله: "يا EJS، خد الـ Array ده وارسمهولي في صفحة الـ Shop".
5. **View:** يولد الـ HTML النهائي ويبعته للـ **Controller**.
6. **Controller:** يبعت الـ HTML ده لليوزر عن طريق `res.send()` أو الـ Render اللي حصل.

💡 ليه الـ MVC ده "اختراع"؟

1. **التخصص (Separation of Concerns):** لو عندك مشكلة في شكل الصفحة، هتروح للـ View. لو الداتا مش بتتحفظ، هتروح للـ Model. مش هتدور في كود المشروع كله.
2. **إعادة الاستخدام (Reusability):** ممكن تستخدم نفس الـ Model عشان تبعت داتا لصفحة ويب (HTML) أو لموبايل أبليكيشن (JSON API).
3. **التطوير الجماعي:** ممكن "ديزاينر" يشتغل على ملفات الـ View، و "باك إند ديفيلوبر" يشتغل على الـ Model في نفس الوقت من غير ما يلبسوا في بعض.

1️⃣ لغم المسار (The Path Logic)

JavaScript

```
const p = path.join(
  path.dirname(require.main.filename),
  "data",
  "products.json"
);
```

- `require.main.filename` : ده بيجيب لك المسار بتاع ملف التشغيل الأساسي (اللي هو `app.js`).
- `path.dirname(...)` : بياخد الفولدر اللي فيه الملف ده.
- **الهدف**: نضمن إننا دايماً بنشاور على فولدر اسمه `data` في الـ Root بتاع المشروع، مهما كان الملف اللي بننادي منه الكود ده فين. دي حركة احترافية عشان المسار ميبوظش (Absolute Path).

2️⃣ الـ `addProduct` : (إزاي بنحفظ الداتا؟)

بص يا سيدي، العملية هنا ماشية بنظام **"اقرأ.. عدّل.. اكتب"**:

1. **اقرأ** (`fs.readFile`): بنروح نشوف الملف فيه إيه.
2. **عدّل**: لو مفيش Error، بنحول الـ JSON لـ Array بـ `JSON.parse` وبعدين نعمل `push` للمنتج الجديد (`this`).
3. **اكتب** (`fs.writeFile`): بنحول الـ Array تاني لنص بـ `JSON.stringify` ونرميه في الملف.

3️⃣ الوحش الكبير: الـ (`cb`) Callback Function

ركز هنا جداً، ليه في `fetchAllProducts` إنت بعتّ (Callback) `cb` ؟

JavaScript

```
static fetchAllProducts(cb) { ... }
```

- **المشكلة**: الـ `fs.readFile` دي عملية **Asynchronous**. يعني Node.js مش هيستنى الملف يتقرأ، هو هيكمل كود ويرجع `undefined` فوراً.
- **الحل**: إنت بتقول للموديل: "يا بطل، لما تخلص قراية الملف، خد الداتا دي وروح نفذ الفانكشن (`cb`) اللي الكنترولر هيديها لك".

🧐 ملاحظات "صنايعية" (Critical Code Review):

⚠️ مشكلة الـ `this` والـ `arrow functions`

إنت مستخدم `(err, fileContent) => { ... }` . دي حركة صح جداً، لأن الـ **Arrow Function** بتحافظ على قيمة الـ `this` اللي بتشاور على الأوبجكت بتاع المنتج اللي إنت لسه عامله `new` . لو كنت استخدمت فانكشن عادية، الـ `this` كانت هتضيع جوه الـ `readFile` .

⚠️ لغم الـ `JSON.parse`

لو الملف موجود بس **فاضي** (Empty String)، الـ `JSON.parse` هيضرب Error ويوقع السيرفر.

- **نصيحة**: دايماً اتأكد إن الملف فيه `[]` على الأقل في البداية، أو اعمل `try...catch` حول الـ `JSON.parse` .

🛠️ إزاي الـ Controller هينادي الكود ده؟

بما إننا شغالين بـ Callback، الكنترولر لازم يتغير لشكل "الاستجابة":

JavaScript

```
// في الكنترولر
Product.fetchAllProducts((products) => {
    res.render('shop', {
        prods: products,
        pageTitle: 'Shop',
        path: '/'
    });
});
```

إنت هنا بتبعت الفانكشن اللي الـ Model هيناديها لما يخلص.